

Padrões de commits 📄

De acordo com a documentação do [Conventional Commits](#), commits semânticos são uma convenção simples para ser utilizada nas mensagens de commit. Essa convenção define um conjunto de regras para criar um histórico de commit explícito, o que facilita a criação de ferramentas automatizadas.

Esses commits auxiliarão você e sua equipe a entenderem de forma facilitada quais alterações foram realizadas no trecho de código que foi commitado.

Essa identificação ocorre por meio de uma palavra e emoji que identifica se aquele commit realizado se trata de uma alteração de código, atualização de pacotes, documentação, alteração de visual, teste...

Tipo e descrição 🦄

O commit semântico possui os elementos estruturais abaixo (tipos), que informam a intenção do seu commit ao utilizador(a) de seu código.

- **feat** - Commits do tipo feat indicam que seu trecho de código está incluindo um **novo recurso** (se relaciona com o MINOR do versionamento semântico).
- **fix** - Commits do tipo fix indicam que seu trecho de código commitado está **solucionando um problema** (bug fix), (se relaciona com o PATCH do versionamento semântico).
- **docs** - Commits do tipo docs indicam que houveram **mudanças na documentação**, como por exemplo no Readme do seu repositório. (Não inclui alterações em código).
- **test** - Commits do tipo test são utilizados quando são realizadas **alterações em testes**, seja criando, alterando ou excluindo testes unitários. (Não inclui alterações em código)
- **build** - Commits do tipo build são utilizados quando são realizadas modificações em **arquivos de build e dependências**.
- **perf** - Commits do tipo perf servem para identificar quaisquer alterações de código que estejam relacionadas a **performance**.
- **style** - Commits do tipo style indicam que houveram alterações referentes a **formatações de código**, semicolons, trailing spaces, lint... (Não inclui alterações em código).
- **refactor** - Commits do tipo refactor referem-se a mudanças devido a **refatorações que não alterem sua funcionalidade**, como por exemplo, uma alteração no formato como é processada determinada parte da tela, mas que manteve a mesma funcionalidade, ou melhorias de performance devido a um code review.
- **chore** - Commits do tipo chore indicam **atualizações de tarefas** de build, configurações de administrador, pacotes... como por exemplo adicionar um pacote no gitignore. (Não inclui alterações em código)
- **ci** - Commits do tipo ci indicam mudanças relacionadas a **integração contínua** (*continuous integration*).

- **raw** - Commits do tipo raw indicam mudanças relacionadas a arquivos de configurações, dados, features, parâmetros.
- **cleanup** - Commits do tipo cleanup são utilizados para remover código comentado, trechos desnecessários ou qualquer outra forma de limpeza do código-fonte, visando aprimorar sua legibilidade e manutenibilidade.
- **remove** - Commits do tipo remove indicam a exclusão de arquivos, diretórios ou funcionalidades obsoletas ou não utilizadas, reduzindo o tamanho e a complexidade do projeto e mantendo-o mais organizado.

✂ Como instalar o arquivo **commit-msg.sh** para validar mensagens de commits com conventional commits

Passo 1: Certifique-se de que o Git está instalado 🌟

Antes de tudo, verifique se o Git está instalado na sua máquina. Abra o terminal e execute:

```
git --version
```

Se você receber uma versão do Git como resposta, está tudo certo! Caso contrário, baixe e instale o Git aqui: [Git Downloads](#).

Passo 2: Localize o arquivo **commit-msg.sh** 📁

O arquivo **commit-msg.sh** deve estar disponível no repositório do seu projeto ou em um diretório específico. Certifique-se de que ele está acessível. Se não estiver, faça o download ou clone o repositório onde ele está localizado.

Por exemplo:

```
git clone https://github.com/seu-repositorio/projeto.git  
cd projeto
```

Passo 3: Crie o diretório **.git/hooks** (se ainda não existir) 📁

Os hooks do Git ficam no diretório **.git/hooks**. Verifique se ele existe no seu projeto:

```
ls -la .git/hooks
```

Se o diretório não existir, crie-o:

```
mkdir -p .git/hooks
```

Passo 4: Copie o arquivo `commit-msg.sh` para o diretório `.git/hooks` 📄

Copie o arquivo `commit-msg.sh` para o diretório `.git/hooks` e renomeie-o para `commit-msg` (sem extensão):

```
cp caminho/para/commit-msg.sh .git/hooks/commit-msg
```

Nota: Substitua `caminho/para/commit-msg.sh` pelo caminho real do arquivo.

Passo 5: Dê permissão de execução ao script ✅

Para que o Git possa executar o script, você precisa dar permissão de execução:

```
chmod +x .git/hooks/commit-msg
```

Passo 6: Teste o hook de commit 🖥️

Agora, tente fazer um commit no seu projeto. Por exemplo:

```
git add .  
git commit -m "feat: adicionar funcionalidade xyz"
```

Se a mensagem de commit seguir o padrão **Conventional Commits**, o commit será aceito. Caso contrário, o hook irá bloquear o commit e exibir uma mensagem de erro.

Passo 7: Personalize o script (opcional) 🎨

Se necessário, abra o arquivo `.git/hooks/commit-msg` em um editor de texto e personalize as regras de validação para atender às necessidades do seu projeto.

Recomendações 🎉

- Adicione um tipo consistente com o título do conteúdo.
- Recomendamos que na primeira linha deve ter no máximo 4 palavras.
- Para descrever com detalhes, usar a descrição do commit.
- Usar um emoji no início da mensagem de commit representando sobre o commit.
- Os links precisam ser adicionados em sua forma mais autêntica, ou seja: sem encurtadores de link e links afiliados.

Complementos de commits 🖥️

- **Rodapé:** informação sobre o revisor e número do card no Trello ou Jira. Exemplo: Reviewed-by: Elisandro Mello Refs #133
- **Corpo:** descrições mais precisas do que está contido no commit, apresentando impactos e os motivos pelos quais foram empregadas as alterações no código, como também instruções essenciais para

intervenções futuras. Exemplo: see the issue for details on typos fixed.

- **Descrições:** uma descrição sucinta da mudança. Exemplo: correct minor typos in code

Padrões de emojis 🇧🇷

Tipo do commit	Emoji	Palavra-chave
Acessibilidade	♿️ :wheelchair:	
Adicionando um teste	✅ :white_check_mark:	test
Atualizando a versão de um submódulo	⬆️ :arrow_up:	
Retrocedendo a versão de um submódulo	⬇️ :arrow_down:	
Adicionando uma dependência	➕ :heavy_plus_sign:	build
Alterações de revisão de código	👉 :ok_hand:	style
Animações e transições	🌀 :dizzy:	
Bugfix	🐛 :bug:	fix
Comentários	💡 :bulb:	docs
Commit inicial	🎉 :tada:	init
Configuração	🔧 :wrench:	chore
Deploy	🚀 :rocket:	
Documentação	📖 :books:	docs
Em progresso	🚧 :construction:	
Estilização de interface	💄 :lipstick:	feat
Infraestrutura	🧱 :bricks:	ci
Lista de ideias (tasks)	👉📅 :soon:	
Mover/Renomear	🚚 :truck:	chore
Novo recurso	✨ :sparkles:	feat
Package.json em JS	📦 :package:	build
Performance	⚡️ :zap:	perf
Refatoração	♻️ :recycle:	refactor
Limpeza de Código	🧹 :broom:	cleanup
Removendo um arquivo	🗑️ :wastebasket:	remove
Removendo uma dependência	➖ :heavy_minus_sign:	build
Responsividade	📱 :iphone:	
Revertendo mudanças	💣 :boom:	fix

Tipo do commit	Emoji	Palavra-chave
Segurança	🔒 :lock:	
SEO	🔍 :mag:	
Tag de versão	🔖 :bookmark:	
Teste de aprovação	✅ :heavy_check_mark:	test
Testes	🧪 :test_tube:	test
Texto	📝 :pencil:	
Tipagem	🏷️ :label:	
Tratamento de erros	🏠 :goal_net:	
Dados	🗂️ :card_file_box:	raw

💻 Exemplos

Comando Git	Resultado no GitHub
<code>git commit -m ":tada: Commit inicial"</code>	🎉 Commit inicial
<code>git commit -m ":books: docs: Atualização do README"</code>	📖 docs: Atualização do README
<code>git commit -m ":bug: fix: Loop infinito na linha 50"</code>	🐛 fix: Loop infinito na linha 50
<code>git commit -m ":sparkles: feat: Página de login"</code>	✨ feat: Página de login
<code>git commit -m ":bricks: ci: Modificação no Dockerfile"</code>	🧱 ci: Modificação no Dockerfile
<code>git commit -m ":recycle: refactor: Passando para arrow functions"</code>	♻️ refactor: Passando para arrow functions
<code>git commit -m ":zap: perf: Melhoria no tempo de resposta"</code>	⚡ perf: Melhoria no tempo de resposta
<code>git commit -m ":boom: fix: Revertendo mudanças ineficientes"</code>	💣 fix: Revertendo mudanças ineficientes
<code>git commit -m ":lipstick: feat: Estilização CSS do formulário"</code>	💄 feat: Estilização CSS do formulário
<code>git commit -m ":test_tube: test: Criando novo teste"</code>	🧪 test: Criando novo teste
<code>git commit -m ":bulb: docs: Comentários sobre a função LoremIpsum()"</code>	💡 docs: Comentários sobre a função LoremIpsum()

Comando Git	Resultado no GitHub
<code>git commit -m ":card_file_box: raw: RAW Data do ano aaaa"</code>	 raw: RAW Data do ano aaaa
<code>git commit -m ":broom: cleanup: Eliminando blocos de código comentados e variáveis não utilizadas na função de validação de formulário"</code>	 cleanup: Eliminando blocos de código comentados e variáveis não utilizadas na função de validação de formulário
<code>git commit -m ":wastebasket: remove: Removendo arquivos não utilizados do projeto para manter a organização e atualização contínua"</code>	 remove: Removendo arquivos não utilizados do projeto para manter a organização e atualização contínua

Principais comandos do Git 📋

- `git clone url-do-repositorio-no-github` - Clona um repositório remoto existente no GitHub para o seu ambiente local.
- `git init` - Inicializa um novo repositório Git no diretório atual.
- `git add .` - Adiciona todos os arquivos e alterações no diretório atual para a área de stage (preparando-os para o commit).
- `git commit -m "mensagem do commit"` - Registra as alterações adicionadas na área de stage com uma mensagem descritiva sobre o que foi modificado.
- `git branch -M main` - Renomeia a branch atual (master) para main. O -M é usado para forçar a renomeação, movendo a branch se necessário.
- `git remote add origin https://github.com/usuario/nome-do-repositorio.git` - Adiciona um repositório remoto chamado origin ao repositório local. Use `https://github.com/usuario` para configurar o repositório remoto com HTTPS ou `git@github.com:usuario` para configurar com SSH.
- `git push -u origin main` - Envia os commits da branch main do repositório local para o repositório remoto origin e define main como a branch padrão para futuros push e pull. O -u (ou --set-upstream) configura a branch upstream para facilitar os próximos comandos git push e git pull e eliminar a necessidade de especificar a branch.
- `git remote add origin git@github.com:usuario/projeto.git` `git branch -M main` `git push -u origin main` - Quando você já tem um repositório local e quer conectá-lo a um repositório remoto no GitHub, adiciona o repositório remoto, renomeia a branch principal para main e envia os commits iniciais.
- `git fetch` - Busca todas as atualizações do repositório remoto sem integrá-las à branch atual. Isso atualiza as referências remotas.
- `git pull origin main` - Atualiza a branch local main com as mudanças do repositório remoto origin. Combina git fetch e git merge.

- `git push --force-with-lease` - Forma mais segura de forçar o envio de alterações locais para o repositório remoto. Verifica se não houve alterações feitas por outros colaboradores desde sua última atualização local, evitando sobrescrever acidentalmente o trabalho de outros.
- `git revert id_do_commit_que_vai_ser_revertido` - Cria um novo commit que desfaz as alterações feitas pelo commit especificado, preservando o histórico. Útil para desfazer mudanças de forma segura sem reescrever o histórico.
- `git reset --hard id_do_commit_anterior_ao_que_vai_ser_apagado` - Redefine o repositório para o estado do commit especificado, apagando todas as mudanças feitas após esse commit. Ideal para uso local. Para sincronizar remotamente, use `git push --force-with-lease` posteriormente.
- `git commit --amend -m "mensagem_reescrita"` - Altera a mensagem do último commit. Após usar este comando, sincronize remotamente com `git push --force-with-lease`.
- `git cherry-pick HASH_DO_COMMIT` - Utilizado para obter um commit específico. Exemplo de uso: Imagine que você tenha duas branches (main) e (develop) e na segunda você tem 3 commits mas deseja apenas pegar o primeiro commit dela, com o uso de cherry-pick você pode.
- `git switch <branch>` - Alterna para uma branch diferente no repositório local. Use `git switch -c <branch>` para criar e alternar para uma nova branch.

Glossário 📖

- `fork` - Cópia de um repositório para a sua própria conta no GitHub. Isso cria um novo repositório em sua conta que é independente do original, permitindo que você faça alterações sem afetar o repositório original.
- `issues` - Ferramenta usada para gerenciar tarefas, pedidos de novos recursos e correções de bugs em projetos de código aberto. As issues devem ser descritas e listadas, permitindo aos colaboradores discutirem e rastrearem o progresso das mesmas.
- `pull request` - Mecanismo usado para submeter alterações propostas ao repositório original. Um pull request é uma solicitação para que os mantenedores do projeto revisem e potencialmente incorporem as alterações. O pull request passará por um processo de avaliação e pode ser aceito ou rejeitado.
- `gist` - Ferramenta que permite o compartilhamento de trechos de código sem a necessidade de criar um repositório completo. Gists podem ser compartilhados publicamente ou de forma privada.

Contribuição ✨

Ajude a comunidade tornando este projeto ainda mais incrível. Leia como contribuir clicando [aqui](#) e a [licença](#). Estou convencido de que juntos alcançaremos coisas incríveis!

Disponível para freelas - #OpenToWork 🚀

Atualmente estou aberto para oportunidades como Desenvolvedor Frontend e UI/UX Designer. Se você precisa de alguém para transformar ideias em interfaces bonitas, responsivas e funcionais, podemos trabalhar juntos!

📧 Entre em contato: ✉ iuricold99@gmail.com 💼 [Linkedin](#)